QUESTION 1

How are discrepancies caught?

Every vendor reply is checked against two references: **what we asked for** (the RFQ) and **what the vendor actually wrote** (their email or PDF). 28 small rules compare those against **what the system extracted**. Every difference becomes a flag with proof, a severity and a next step.

REFERENCE 1

What we asked for

The RFQ email: 10 parts, NSNs, 27 quantity breaks, the required fields

REFERENCE 2 · THE EVIDENCE

What the vendor wrote

The email body or PDF page; the latest revision only

BEING CHECKED

What was extracted

`products[] → lines[]`: a claim, which can be wrong

Check A · Extraction vs the vendor's words

Catches **data errors**: a price the email states but the data dropped, a changed value, a wrong FAT flag, details invented on a no-bid, a line on only one side.

Check B · Quote vs the request

Catches **business discrepancies**: no-bids, missing or extra quantity breaks, alternate or look-alike parts, NSN, COO, certs, currency, payment terms, expiry, minimum order.

How a single check works

1. **Parse both texts into items.** RFQ lines become requested parts with their quantity breaks. Vendor text becomes item blocks and price lines, each with its position in the text.
2. **Match.** Each extracted product is paired with its RFQ item: exact, an alternate (the vendor wrote "ALT for X", or the same NSN), or not requested. Each line is paired with the vendor's line of the same quantity.
3. **Compare with a rule.** Each rule is a small function over one pair. For example: *missing breaks = requested quantities − quoted quantities.*
4. **Raise a flag.** It carries a severity, where the evidence is in the email, a one-click fix when the email has the right value, and a sentence for the vendor follow-up.

RULE (EXAMPLES OF 28)	WHAT IT COMPARES	CAUGHT IN THIS DATA	SEVERITY
Dropped value	The vendor's price line vs the extracted line	Heritage MS27039-1-08 @100: the email says $1.74 with an 18-day lead; the data is blank	Blocker
Wrong FAT flag	The vendor's FAT note vs `isFat`	Apex @100 is marked FAT, but the PDF never mentions FAT	Blocker
Missing break	Requested quantities vs quoted quantities	Heritage item 1: we asked for 100/250/500; they quoted 100 and 500	Warning
Alternate offered	The RFQ part number vs the quoted part	Heritage quotes NAS1149F0832P-ALT for item 2 (NAS1149F0832P)	Warning
Look-alike part	An unrequested part vs requested parts (within 2 characters)	Apex adds NAS1149F0**3**32P, one character from NAS1149F0**8**32P	Warning
Compliance	COO and certifications	Northstar: MS9556-04 has "No cert"; NAS1149F0832P has no COO	Blocker
Terms	Currency, payment terms, validity	Northstar quotes in EUR with prepayment; every quote has passed its validity date	Warning

Severity decides what happens next. A Blocker must be fixed, or accepted with a written reason, before a quote can be marked verified. A Warning needs a decision or a vendor follow-up. Info is good to know. On this file: 10 blockers, 43 warnings, 10 info.

Why rules in code, not an LLM? These checks are structural, so code is exact, instant, free and explainable. A golden test pins every result on the real file. An LLM can be added later for free-form wording.

How do you see the email and verify every number?

Every extracted value is **linked to the exact characters** in the vendor's email or PDF that it came from. The review screen shows the email **beside** the extracted quote. Click any value and its source line is highlighted. A status says whether the two match, and one click fixes a mismatch from the email.

VENDOR'S EMAIL · HERITAGE · MAY 14 — QUOTE SOURCE

```
1. MS27039-1-08
   NSN 5305-00-984-6210
   product note: FAT applies to the first line shown.
      qty 100: USD 1.74 ea / 174.00 ext / 18d
      qty 500: USD 1.62 ea / 810.00 ext / 27d
```

⟷ span chars 479–518

EXTRACTED QUOTE · ITEM 1 · REQUESTED 100 / 250 / 500

QTY	UNIT	EXT	LEAD	STATUS
100	—	—	—	Dropped: email says $1.74 · 18d · Apply email value
250	*not quoted*			Missing break
500	1.62	810.00	27 d	✓ Matches email

1 Keep every message

Each email body and PDF page is stored as a document, labeled RFQ, quote source, superseded or no quote. Quoted history ("Original Message") is shown dimmed and never parsed.

2 Remember where each value came from

While parsing, every value records a **span**: the document, plus start and end character. Here that is Heritage's email, chars 479 to 518.

3 Pair and compare

Each extracted line is paired with the vendor's line of the same quantity. If price, extended price, currency, lead time and FAT all agree, it shows ✓ **Matches email**. Otherwise it becomes a blocker.

4 Show them side by side

The Source pane renders the email as plain text, cut at the span boundaries. Green means it matches. Amber or red means flagged. An outline marks the selected line.

5 Click to verify

Click any row, number or flag, and the email scrolls to that exact sentence and outlines it. No searching through the thread.

6 Fix from the source

Apply email value copies the email's numbers into an override. It is logged and can be undone. The flag clears, and the row shows "from email".

The customer sees the same proof. In the Customer view, clicking any price opens the vendor's own line, highlighted, with a plain-language status: Verified · Corrected from email · Open issue.

Why spans? They are exact and cheap, and they work the same for email bodies and PDF pages. Rendering as plain-text segments means no HTML injection. In production, the AI extractor would return these spans as citations, so free-form replies get the same click-to-verify.

From one JSON file to three screens, in six steps

Each step builds a new typed structure from the one before. Nothing is edited in place; a click only changes a small overlay of decisions, and step 6 runs again.

	WHAT HAPPENS	WHAT IT PRODUCES	WHY

1	**Load the JSON**
	Import `state.json`, using TypeScript types that mirror its shape exactly.
		`RawState`
		`vendors[] → emails[] → attachments[].pages[]`
		`vendors[] → products[] → lines[]`
		3 vendors · 7 emails · 33 products · 62 lines
			A misspelled field fails at compile time, not in front of a customer. The raw data is read-only, so it is never changed by mistake.

2	**Read what we asked for** Parse the outbound RFQ email. Each line "1. MS27039-1-08 | NSN … | qty breaks 100/250/500" becomes one item.	`RfqItem { no, partNumber, nsn, qtys: number[], span }` 10 items, 27 quantity breaks, due May 20	The request is the yardstick every quote is measured against. It is kept as an ordered list, because RFQ order drives every table in the UI.

3	**Read what the vendor said**
	Turn each email body and PDF page into a document. Keep the newest message with a quote, skip quoted history, then parse item blocks and price lines.
		`SourceDoc { key, role, text }`
		`SourceItem { partNumber, altFor?, noBid, lines, … }`
		`SourceLine { qty, currency, unitCost, price, leadTime, span }`
		Heritage's "qty 100: USD 1.74 ea…" sits at chars 479 to 518
			The vendor's own words are the evidence. Every value keeps its exact position in the text (its span), so the UI can highlight where it came from.

4	**Pair the data with the evidence**
	Match each product to its RFQ item (exact, alternate or not requested) and its source block; match lines by quantity.
		`ProductReview { product, kind, rfqNo, source, lines }`
		`LineReview { line, source, included, status }`
		NAS1149F0832P-ALT is an alternate for item 2
			Map lookups by part number and by quantity make pairing instant. Part numbers are normalized, so formatting differences do not break a match.

5	**Check everything** Run 28 small rules over the pairs: data vs email, quote vs request, compliance and terms.	`Flag { id, rule, severity, detail, evidence: Span, fix?, followUp? }` MISSING_QTY_BREAK · item 1 @250 · warning	Each rule is a pure function, easy to test and to explain. A flag carries its proof, a one-click fix, and the sentence to send the vendor.
6	**Build what the screens show** `buildReview(state, overlay)` applies decisions, re-runs the rules and assembles what the screens need.	`ReviewModel { vendors: VendorReview[], compare: CompareRow[], po: PoPlan }` Heritage: 3 blockers · 9 warnings · 3 info	One derived model is the single source of truth. Screens only display it, so the Ops, Customer and Vendor views can never disagree.

Then The **Ops**, **Customer** and **Vendor** views render the model. Every click (apply, accept, exclude) changes only the overlay; step 6 runs again, and every view updates.

Verify every number against the vendor's own words, and show each stakeholder their view

A frontend-only app built with Vite, React and TypeScript. It reads `state.json`: one RFQ for 10 parts and 27 quantity breaks, replies from 3 vendors, and an AI extraction of each reply. It treats the extraction as a **claim** and the vendor's text as the **evidence**. It flags every gap against the RFQ, and shows each stakeholder their own view.

1

See the email, verify the quote

The vendor's email or PDF sits beside the extracted quote. Click any number and its exact source line lights up. If the data disagrees with the email, it is flagged, and one click applies the email's value.

2

Auto-flag every discrepancy

28 rules catch missing breaks, alternates, look-alike parts, COO, certs, currency, expiry and extraction errors. Each flag is a blocker, a warning or info. Blockers must be cleared before a quote can be marked verified.

3

Easy to use, per stakeholder

Tables follow the RFQ, so gaps show as rows. Severity is always an icon, a word and a color. Fixes take one click and can be undone.

VIEW 1

SalesPatriot Ops

Does the work. Verifies each reply against its email, fixes extraction errors, resolves flags, sends follow-ups, and prepares POs.

VIEW 2

Customer

Decides what to bid. Sees the recommended vendor and price for each break, a plain-language trust status, and the source email one click away.

VIEW 3

Vendor

Answers what is missing. Sees only their own request next to their reply, and a checklist of asks. Never sees other vendors or internal flags.

Stack, and why

- **Vite, React and TypeScript (strict):** the fastest reliable 30-minute build. The logic is framework-free, so it ports to Svelte.
- **Tailwind v4:** quick, consistent styling.
- **zustand with localStorage:** a tiny store, with persistence built in.
- **Vitest:** the golden test on the real file.
- **Playwright:** a real-browser check of each view.
- No server, no database, no LLM calls.

In this document

- 1 · How discrepancies are caught
- 2 · How you see the email and verify
- 3 · How it works, in six steps
- 5 · Data structures, and why each one
- 6 · Architecture and code choices
- 7 and 8 · The Ops, Customer and Vendor views
- 9 to 11 · Good to know: back end and SQL; race conditions and scale; errors, security and testing

The data structures that make the parsing work

STEP	STRUCTURE	USED FOR	WHY THIS ONE
3	`Span { docKey, start, end }`	Evidence and highlighting	Character positions point at the exact sentence. The text is split into segments around them, so highlighting needs no HTML injection.
4	`Map<partNumber, SourceItem>`	Pair a product with its text	Instant lookup. Part numbers are trimmed and uppercased first, so formatting differences still match.
4	`Map<qty, SourceLine>`	Pair a line with its text	Quantity is the natural key of a price break.
5	`Set<number> of quoted qtys`	Missing and extra breaks	Requested minus quoted gives the "not quoted" rows. Quoted minus requested gives the extras.
4–5	`MatchKind · Severity · RuleId · Fix`	Kinds of match, flag and fix	Union types. The compiler forces every case to be handled, so there are no misspelled-string bugs.
5	`RULES: Rule[]`	The 28 checks	An array of pure functions. A new rule is one function and one line, and each rule can be tested on its own.
5	`id = vendor:rule:target:qty`	Flags and decisions	A stable id means a decision survives recomputation and page reloads. It also serves as the React key.
6	`Overlay: Record<id, …>`	Fixes, decisions, audit log	Plain JSON, so it saves to localStorage and exports as-is. The source is never mutated, and undo is deleting a key.
6	`CompareRow · cells by vendor id`	Comparison and PO	27 breaks × 3 vendors is a grid. The best cell is the cheapest verified, compliant, exact-part quote.
5	`levenshtein(a, b)`	Look-alike part numbers	A small edit-distance check. It catches NAS1149F0**3**32P against NAS1149F0**8**32P, one character apart.

Worked example: one Heritage line, traced through the steps

```
Step 3  email text   "qty 100: USD 1.74 ea / 174.00 ext / 18d"   (chars 479 to 518)
        SourceLine   { qty: 100, currency: "USD", unitCost: 1.74, price: 174, leadTime: 18, isFat: true }
Step 4  extracted    { quantity: 100, unitCost: null, price: null, currency: "", leadTime: null, isFat: null }
Step 5  flags        EXTRACTION_DROPPED_VALUE and EXTRACTION_FAT_MISMATCH (both blockers), evidence = chars 479 to 518
Click   overlay      lineOverrides[line] = { unitCost: 1.74, price: 174, currency: "USD", leadTime: 18, isFat: true }
Step 6  result       both flags clear · the row shows "from email" · Heritage becomes best on item 1 @100
```

Why regex, not an LLM

The vendor text is line-based and consistent. Regex is exact, instant, testable, and it gives positions. **Trade-off:** free-form replies would not parse; production would have the extractor return the spans.

Revisions

The newest message with a quote wins. Text after "Original Message" is never parsed, so Northstar's obsolete rev 2 prices can't sneak in.

Never crashes

Parsers never throw. Text that can't be parsed shows as "unverified" in grey. Every regex lives in `constants.ts`, with a comment.

The parsed shapes: the RFQ, and a vendor's reply

Each parser turns the raw JSON into shapes built for one job: verification. The RFQ becomes a flat list of requested parts; a reply becomes the vendor's own words, re-read from the email or PDF text. Every field that came from text carries a Span, so a flag can point at the exact sentence.

① THE PARSED RFQ — parseRfq(doc)

```
interface Rfq {
  docKey:   string;     // doc the spans index
  subject:  string;
  sentAt:   string;     // ISO date sent
  dueDate?: string;     // "2026-05-20"
  items:    RfqItem[];  // the 10 parts
}

interface RfqItem {
  no:         number;   // 1..10
  partNumber: string;   // "MS27039-1-08"
  nsn:        string;
  qtys:       number[]; // [100,250,500]
  span:       Span;     // → highlight
}
```

A REPLY DOCUMENT — SourceDoc

```
interface SourceDoc {   // email body OR one PDF page
  key; emailId;
  role; // rfq|quote|superseded|no-quote
  from; fromName; date; subject; text;
  attachment?: { filename; page };
  historyStart?: number; // text after = ignored
}
```

Parsed per vendor

The RFQ is parsed from each vendor's own copy, so a span points into the email that vendor received. Roles sort the thread: the newest doc with a quote is `quote`; Northstar's obsolete rev 2 falls after `historyStart` and is never read.

② THE VENDOR REPLY, PARSED — parseVendorSource(vendor)

```
interface ParsedSource {
  docs:        SourceDoc[];   // emails + PDF pages
  rfq:         Rfq;
  quoteDocs:   SourceDoc[];   // the quote doc(s)
  items:       SourceItem[];  // quote, from text
  respondedAt: string;
  sourceLabel: string;        // "PDF attachment"…
  hasParsedSource: boolean;   // false → warn
}

interface SourceItem {   // one block = the evidence
  no: number; partNumber: string;
  altFor?: string;        // set if an alternate PN
  extraStock: boolean; noBid: boolean;
  nsn?; description?; mfrPartNumber?;
  mfrCage?; coo?: string|null; hazmat?;
  packagingIncluded?; shippingIncluded?;
  certifications?; nreCost?: number|null;
  note?; noteSpan?: Span;
  lines: SourceLine[];    // the priced breaks
  span: Span; headerSpan: Span;
}

interface SourceLine {   // one priced break
  qty: number;      currency: string;
  unitCost: number; price: number;
  leadTime: number; isFat: boolean;
  span: Span;
}
```

Why two layers — the claim and the evidence

The raw `products[]` in `state.json` is the **AI extraction**: a claim, with planted errors (dropped prices, wrong FAT flags, fields invented on no-bid rows). `SourceItem` and `SourceLine` are **ground truth**, re-read from the vendor's own words. The 28 rules diff the two; a mismatch becomes a blocker or warning whose evidence span highlights the source line, and whose one-click fix copies the parsed value over the claim. **Trade-off:** the two parsers are tuned to these reply formats; an unknown layout degrades to "unverified", never a crash.

A pure core, one source of truth, thin views

All business logic is in `src/domain`, plain TypeScript with no React and no clock. The UI renders `buildReview(state, overlay)`. A user action changes only the overlay, and everything else is recomputed from it.

The loop

Click Apply, Accept, Exclude, Include › **Overlay** override or decision, plus an audit entry › **buildReview** memoized on the overlay › **Views** re-render

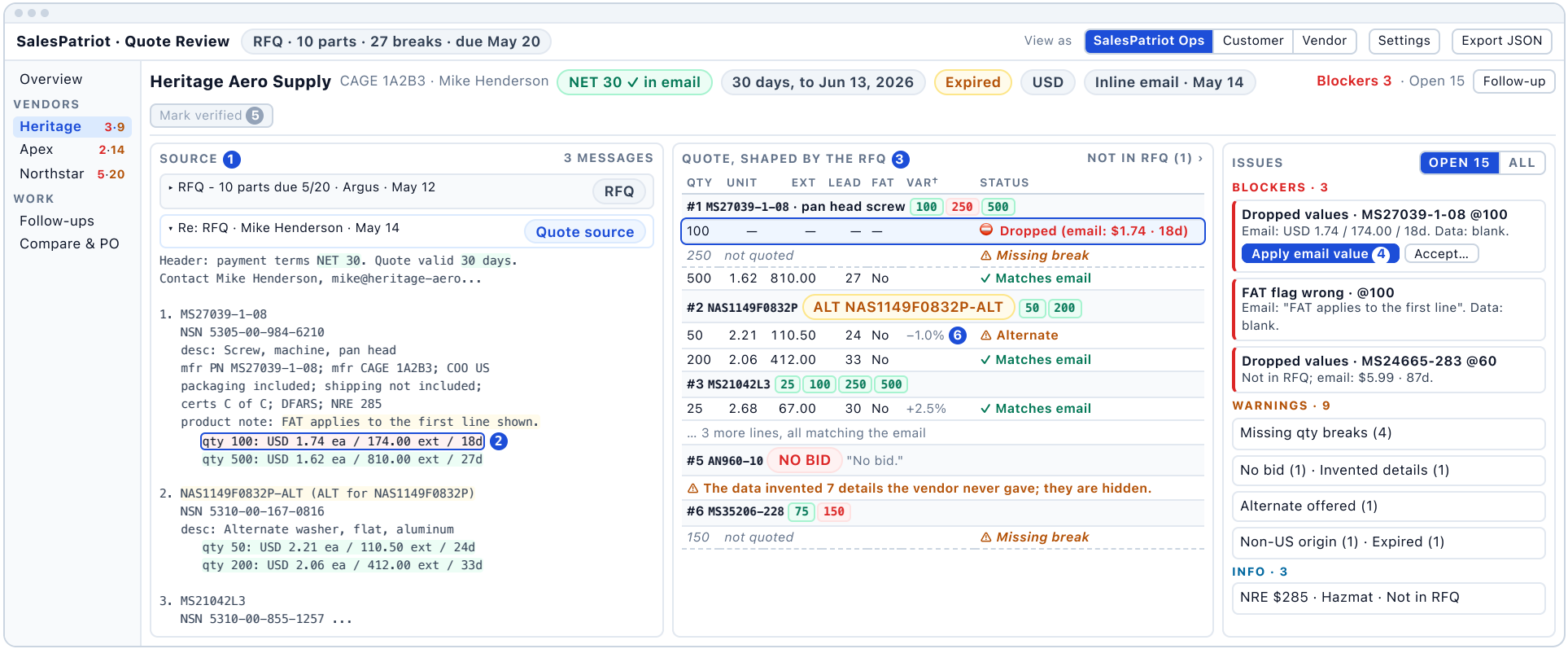
A fix never edits a flag. It overrides a value, and the flag disappears because the data now matches the email.

Files, one responsibility each

`domain/types.ts`	The contract between domain and UI
`domain/constants.ts`	Regexes, tolerances, defaults, with reasons
`domain/parseRfq.ts · parseSource.ts`	Text to typed items and spans
`domain/match.ts`	exact, alternate or unrequested
`domain/rules.ts`	28 rules, one function each
`domain/followUp.ts`	Vendor email and stored-draft check
`domain/compare.ts`	Comparison, trust, PO plan
`domain/review.ts`	buildReview: applies overrides, runs everything
`store/useReview.ts`	Overlay (persisted) and view state
`ui/shell · shared · ops · customer · vendor`	One folder per stakeholder

CHOICE	WHY	TRADE-OFF
Pure domain module	The golden test runs the whole engine on the real file in milliseconds, with no browser. The same code can run in a Node worker or their Svelte app.	A little more upfront design (types first).
Derived state only (`useMemo(buildReview)`)	The UI can't drift from the rules, because it never computes them. No sync bugs between screens.	Full recompute per action. That is O(lines), under 1 ms here.
Overlay, never mutate	Every change is reversible and audited, and the export is the original plus decisions.	Overrides must be re-applied if a new revision arrives.
zustand with persist (versioned, migrate)	A tiny API, with localStorage built in. The storage event re-syncs two open tabs.	Single user, single device.
Deterministic time	Rules read `settings.asOf`, never `Date.now()`, so tests are stable. The UI defaults it to today, which shows every quote as expired.	One more setting.
Severity-gated actions	Blockers must be fixed or accepted with a reason before "Mark verified". Money and compliance stay protected.	Friction; keyboard shortcuts help.
Untrusted text as text nodes	Vendor emails could contain markup. Highlights are React segments, never innerHTML.	No rich-HTML email rendering.
Money as numbers, with tolerances	The prices have two decimals. Comparisons use ±0.01 (extended) and ±0.001 (unit) to avoid float noise.	Production would use decimal or integer cents.
React, not Svelte	The fastest reliable 30-minute build. The domain is framework-free, so it ports to their Svelte stack unchanged.	The UI layer would be rewritten.
Contracts first, 3 agents in parallel	`types.ts`, the store and the shared props are written first, so the domain, Ops UI and Customer/Vendor UI are built at the same time.	Integration time is reserved (minutes 15 to 21).
Tests that pin behavior	A golden test asserts every planted issue on the real file, with exact counts. Playwright walks each stakeholder's path, with a clean console.	One real file can overfit; add property tests later.
Code bar	A header comment per file, JSDoc on exports, names that say what things are, small functions, no magic values, strict TypeScript, and lint clean.	A few minutes of the timebox.

Verify a reply in one screen: source, quote and issues side by side



1. **Source next to the data.** Every message in date order. The RFQ is collapsed. The quote source is labeled. Superseded history is dimmed.
2. **Click any number to see where it came from.** The exact line is outlined and scrolled into view. Green tint means a value matches the email; amber marks evidence behind a flag.
3. **Shaped by the request.** Grouped by RFQ item, with break chips. An empty row for each unquoted break, so gaps are visible. No-bid rows quote the vendor.
4. **One-click fix from the email.** It writes an override with an audit entry. The flag clears on recompute, and the row shows "from email". Undo is available.
5. **"Mark verified" is gated.** Disabled while blockers are open; the tooltip lists them. Accepting a blocker requires a written reason.
6. **Honest about unknowns.** `variance` is not in any vendor message, so it carries a dagger and an open question.

Why three columns? Verification means looking at two things at once. Putting issues in their own column keeps the to-do list visible while you read. Below 1440px, issues move into a tab.

The customer sees what to trust; the vendor sees what to send

SalesPatriot · Quote Review — View as: SalesPatriot Ops | **Customer** | Vendor

Quote summary · RFQ - 10 parts due 5/20

10 · 27	25	2	Jun 13
parts · price breaks	verified, compliant source (after Ops fixes)	need a vendor follow-up	earliest quote expiry: reconfirm

PART	QTY	RECOMMENDED	UNIT	LEAD	STATUS	OTHERS
MS27039-1-08	100	Heritage	$1.74	18 d	Corrected	2 ›
	250	Apex	$1.84	34 d	Verified	0
	500	Heritage	$1.62	27 d	Verified	1 ›
NAS1149F0832P	50	Apex	$2.42	31 d	Verified	2 ›
	200	—			No compliant source	2 ›
MS21042L3	25	Heritage	$2.68	30 d	Verified	1 ›
MS17985C411	15	Northstar	€7.04 ≈$7.74	80 d	Verified	0
MS9556-04	192	—			No compliant source	1 ›

… 19 more rows · **Verified** = matches the vendor's email · **Corrected** = fixed from the email by Ops

EVIDENCE · HERITAGE · EMAIL OF MAY 14 ✕

```
1. MS27039-1-08
   NSN 5305-00-984-6210
   product note: FAT applies to
   the first line shown.
      qty 100: USD 1.74 ea / 174.00
      ext / 18d
      qty 500: USD 1.62 ea / 810.00
      ext / 27d
```

WHAT HAPPENED

The price was missing in the system. SalesPatriot Ops corrected it from this email on Oct 12.

GOOD TO KNOW

First-article test applies to this line. One-time NRE $285. Quote validity ended Jun 13: reconfirm before ordering.

Customer summary. It answers "can I bid on these numbers?" Each price has a trust status in plain words, and any price opens the vendor's own email with that line highlighted. No rule IDs and no extraction jargon. It is roomier than the Ops view: fewer columns, larger numbers.

Trade-off: how much should the customer see?

Showing every flag would bury the decision. Showing none would ask for blind trust. The middle path: a recommendation with a trust chip, the evidence on demand, and a short plain-language "what happened". Ops keeps the full detail.

Vendor portal preview — Ops | Customer | **Vendor: Apex ▾**

RFQ from Argus Defense

Sent May 12 · due May 20 · your reply received May 15

#	PART	REQUESTED BREAKS VS YOUR QUOTE
1	MS27039-1-08	100 $1.95 · 250 $1.84 · 500 $1.74
2	NAS1149F0832P	50 $2.42 · 200 needed
3	MS21042L3	25 · 100 · 250 · 500
6	MS35206-228	You offered MS35206-228-AC (under review)
7	MS9556-04	12 · 48 · 96 · 192
9	MS17985C411	No bid
10	AN3-12A	25 · 100 needed

Items 4, 5 and 8 are complete · Not requested: NAS1149F0332P (not included)

WHAT WE STILL NEED · 8

- ☐ Item 2: add the 200 pc break
- ☐ Item 3: add the 25 and 500 pc breaks
- ☐ Item 6: quote MS35206-228 exactly, or send the spec sheet for -AC
- ☐ Item 7: add the 48 and 192 pc breaks
- ☐ Item 9: can you quote MS17985C411?
- ☐ Item 10: add the 100 pc break
- ☐ NAS1149F0332P is not on our RFQ: confirm it is not a substitute
- ☐ Validity ended Jun 29: confirm the pricing holds

Vendor view. It answers "what do you still need from me?" Their request against their reply, plus a checklist. The prototype shows it as a preview; in production it would be a portal.

Never shown to a vendor

- Other vendors, or any competing price.
- Internal flags, such as Apex's invented FAT flag (our data error, not theirs) or COO judgments.
- `variance`, decisions, notes and the audit trail.

Why. Sharing a competitor's price breaks trust and can harm the negotiation. Sharing our extraction errors makes us look careless and does not help them answer.

Same rules behind an API; Postgres as the system of record

Email webhooks
Gmail or Graph; signature verified; dedupe on Message-ID

›

Postgres + S3
Messages and threads in Postgres; PDFs in S3 by sha256

›

Queue and workers
OCR, then LLM extraction with cited spans and confidence

›

Rules package
The prototype's `src/domain`, unchanged

›

API: REST + SSE
Builds each role's view and filters it on the server

›

UI, outbox, ERP
Svelte views; follow-ups sent once; POs pushed idempotently

KEY TABLES (BLUE = UNIQUE CONSTRAINT, WHICH DOUBLES AS DEDUPE OR IDEMPOTENCY)

rfqs · rfq_items
id · tenant_id · due_date
line_no · part_number · nsn · qty_breaks int[]
UQ (rfq_id, line_no)

vendor_quotes
rfq_id · vendor_id · status
current_revision_id · version
UQ (rfq_id, vendor_id)

messages · attachments
provider_message_id · thread_key
auth_results jsonb · s3_key · sha256
UQ (tenant_id, provider_message_id)

quote_revisions
vendor_quote_id · rev_no
raw_extraction jsonb · extractor_version
UQ (vendor_quote_id, rev_no)

quote_lines
revision_id · rfq_item_id · qty
unit_cost numeric(12,4) · currency char(3)
lead_days · is_fat · evidence jsonb

flags · flag_decisions
rule_id · rules_version · fingerprint
status · note · value_hash · version
UQ (vendor_quote_id, fingerprint)

line_overrides
line_id · field · value
evidence · by · at

outbox · po_drafts
payload jsonb · sent_at · attempts
fx_snapshot jsonb · erp_ref
UQ idempotency_key

audit_log
actor · action · target
before / after jsonb · at
append-only (no UPDATE or DELETE)

Prototype to production.

- `src/domain` becomes a shared package.
- The overlay becomes `line_overrides` and `flag_decisions`.
- localStorage becomes Postgres.
- The storage event becomes SSE.
- `View as` becomes RBAC. The vendor's payload never contains internal fields.

Why Postgres (SQL)

- **The data is relational.** RFQ, then items, then breaks, matched against vendor quotes, revisions and lines. The comparison and coverage views are joins and group-bys.
- **Exact money.** `numeric` plus ISO currency codes, never floats.
- **Constraints do safety work.** Unique keys dedupe webhooks and make retries idempotent. Foreign keys stop orphan lines.
- **Transactions.** A decision, its audit row and its outbox row commit together, or not at all.
- **Row-level security** isolates tenants.
- **It is SalesPatriot's stack:** Postgres with hand-written SQL.

Why not NoSQL as the main store

A document per quote (Mongo or DynamoDB) matches the JSON shape and scales writes easily. But cross-vendor comparison, unique constraints, multi-row transactions and reporting all get harder. Eventual consistency is risky for prices and approvals.

Cost of choosing Postgres: sharding work at very large scale (Citus, or a cell per big customer), and managed migrations.

Where non-relational still fits

- **JSONB** for raw LLM output and evidence spans, whose shapes keep changing.
- **S3** for PDFs: private, content-addressed.
- **Redis** (optional) for rate limits and SSE fan-out.
- **OpenSearch**, later, for full-text search over years of email.

API trade-offs.

- **Modular monolith over microservices:** one deploy for a small team.
- **REST over GraphQL:** few clients, and ETags plus idempotency keys are simple to reason about.
- **SSE over websockets:** updates only flow from server to client.

No lost updates, no double sends, no duplicate POs, and scale where it hurts

SCENARIO	MECHANISM, AND WHY
Webhook redelivered, or the same email twice	`UQ(tenant_id, provider_message_id)` with `INSERT … ON CONFLICT DO NOTHING`. The database guarantees exactly one row, so no app-side locking is needed.
Emails out of order (rev 2 lands after rev 3)	Order revisions by `sent_at`. `current_revision_id` only moves forward, via a conditional UPDATE. A per-thread advisory lock is held while processing, so a stale revision can never overwrite a newer one.
Two workers grab one job	`SELECT … FOR UPDATE SKIP LOCKED`, with a lease. Each job runs once, without paying for extraction twice.
Two reviewers edit one quote	Optimistic locking: a `version` column checked with If-Match. A conflict returns 409, which opens a merge view. Decisions are one row per flag, so different flags never collide.
A new revision arrives mid-review	An SSE banner appears. Each decision stores a `value_hash`, and carries over only if the underlying value is unchanged; otherwise it needs another look.
"Mark verified" double-clicked or retried	An Idempotency-Key, plus a conditional transition: `UPDATE … WHERE status='in_review' AND version=$v`. A replay returns the first result.
Follow-up sent twice	Transactional outbox: the follow-up row and the outbox row commit together. A deterministic Message-ID and a per-vendor cooldown stop double sends.
PO creation retried after a timeout	The idempotency key is stored with UNIQUE. Check the ERP for the reference before creating. A nightly reconcile job catches the rest.
Rules or FX change later	`rules_version` on every flag. An FX snapshot is stored on the comparison and PO, so awards never flip silently.

Why optimistic, not pessimistic locking

Only a few reviewers touch an RFQ, so conflicts are rare. Pessimistic locks ("Janet is editing") block people and leak when tabs close. Optimistic checks almost never fire, and when they do, a merge view is cheaper than a lock manager.

Policy: READ COMMITTED, plus row locks at state transitions. SERIALIZABLE only for award allocation, with a retry. Never hold a transaction open across an LLM or ERP call.

Trade-off: one merge screen to design and test.

Scalability: what changes

The bottleneck is LLM extraction (cost and provider limits), not the rules, which take microseconds per reply. Rough assumption to check: 50 customers × 500 RFQs a day × 4 vendors is about 100k replies a day, or 1.5M lines.

1x	Postgres queue (SKIP LOCKED), one or two worker pools, review model cached per revision
10x	SQS and autoscaling. An LLM gateway with a content-hash cache. PgBouncer, a read replica, monthly partitions. Virtualized tables in the UI.
100x	Ordered queues per thread. Shard by tenant, or one cell per big customer. OpenSearch. Precomputed read models.

Why boring first: a Postgres queue avoids another system on day one. Move to SQS when queue age p95 passes 60 s.

In the prototype, the same ideas in miniature: decisions are keyed by stable flag ids, the storage event re-syncs tabs, the persisted store is versioned with migrations, and fixes are overrides that never edit the source.

Fail visibly, trust carefully, test what matters

Error display

Bad input	Inline, under the field. The button stays disabled.
Blocked action	The disabled button's tooltip lists what is left.
Action done	A toast with Undo; an audit entry is written.
Unparseable reply	A yellow banner; values show as "unverified" in grey.
Storage corrupt	A banner, and a fresh start. Export still works.
View crash	An error boundary in that view only, with Reload and Reset.
409 conflict prod	"Updated by Janet: 3 decisions still apply, 1 needs a look", with a diff.
Network, 429, job failure prod	A banner with automatic retry. Failed jobs show Retry and alert Ops.

Why: reviewers handle money, so a silent failure is the worst outcome. Every message says what happened and what to do next. Input is never lost.

Trade-off: more UI states to design and test.

Security and bots

- **SSO with MFA**, no passwords. That removes credential stuffing and most login bots in one move.
- **RBAC and tenant RLS.** Roles are filtered on the server, so the vendor payload never contains internal data.
- **Check SPF, DKIM and DMARC on vendor mail.** A spoofed "updated price" email raises an "Unverified sender" blocker. A fake quote is the real threat.
- **Rate limits** per user, tenant and IP, behind a WAF with bot management. Turnstile and signed, expiring links on the vendor portal.
- **Signed webhooks**: verify the signature, reject stale ones, stay idempotent.
- **Untrusted content.** Vendor text renders as text nodes, never HTML, with a CSP. Extraction treats email as data, with schema-constrained output, which guards against prompt injection.
- **Append-only audit log.** Two-person approval for large POs. Encryption, a secrets manager, CUI-aware hosting.

Trade-off: setup with each customer's identity provider. **In the prototype:** there is no server, text-node rendering, encoded mailto links, nothing secret stored.

Testing

Golden test built	Every planted issue in state.json, with exact counts. Rev 2 prices are never used.
Unit built	Each rule alone, table-driven, including edge cases.
End to end built	Playwright walks each stakeholder's path, with a clean console.
Property later	Parsers never throw; spans stay in bounds.
DB integration later	Constraints, RLS, idempotent replays (Testcontainers).
Concurrency later	10 parallel verifies give 1 transition.
Extraction evals later	Reviewer corrections become labeled cases. CI blocks a regression.

Why: the rules are the product, so the tests pin their exact output on real data. The evals keep the LLM honest over time.

Trade-off: one real file can overfit. Property tests and growing eval sets fix that.